OPERATIONAL THREAT HUNTING & INCIDENT RESPONSE PROTOCOL // LEVEL 3

# TACTICAL CYBER DEFENSE PLAYBOOK

Compromise Assessment, Kernel Rootkit Hunting & RCE Mitigation Field Manual

| | |
|---|---|
| **Document Version:** 3.0-PRODUCTION | **Target Architectures:** Windows x64, Linux VPS/cPanel, Android & iOS |
| **Classification:** RED/BLUE TEAM TACTICAL MANUAL | **Author / Role:** Principal Red/Blue Systems Security Architect |
| **Toolkit Components:** 3 Audit Scripts + Portal Server (portal_server.py) | **Remediation Portal:** http://127.0.0.1:8888 (Local Auto-Spawn) |

**[ALERT] CRITICAL OPERATIONAL DOCTRINE**

Never trust a compromised operating system to report its own compromise. Kernel-level rootkits, Bring-Your-Own-Vulnerable-Driver (BYOVD) exploits, and Direct Kernel Object Manipulation (DKOM) can systematically blind live query tools (Task Manager, Get-Process, netstat, ps). This playbook integrates rigorous live heuristic telemetry with Out-of-Band (OOB) offline verification to ensure complete tactical visibility.

## 1. Threat Architecture & The Reality of Rootkits

Modern advanced persistent threats (APTs), commercial spyware cartels (e.g., NSO Group, Cytrox, Intellexa), and sophisticated ransomware syndicates (e.g., BlackCat, LockBit) operate predominantly at ring boundaries below traditional security controls. Understanding the mechanics of system visibility is paramount:

### The Dual-Ring Execution Barrier:

- **User Mode (Ring 3):** Standard applications, services, and default administrative tools run in Ring 3. They rely on Windows API subsystems (NTDLL, KERNEL32) or Linux glibc system call wrappers to interface with the kernel. Userland rootkits manipulate the dynamic linker (/etc/ld.so.preload, LD_PRELOAD) or inject hooks into standard memory spaces to filter out malicious files, sockets, and processes.
- **Kernel Mode (Ring 0):** The operating system kernel, hardware abstraction layer (HAL), and kernel drivers execute with unrestricted hardware access. A rootkit executing in Ring 0 possesses total sovereignty over system memory and execution state.

### How Kernel Rootkits Blind Live Operating System Scanners:

1. **Direct Kernel Object Manipulation (DKOM):** Windows tracks running processes via the doubly-linked list ActiveProcessLinks within the EPROCESS kernel structure. A kernel rootkit unlinks its malicious process from this chain. The thread continues executing on the CPU scheduler, but userland query APIs (EnumProcesses, Process32First/Next, WMI Win32_Process) are blinded.
2. **System Service Descriptor Table (SSDT) & Syscall Filtering:** In Linux and pre-PatchGuard Windows, rootkits hook syscall dispatch tables. On modern 64-bit systems, threat actors employ hypervisor-based virtualization (Type-1/Type-2 VMCS shadowing) or filter kernel notification routines (PsSetCreateProcessNotifyRoutine) to intercept security callbacks.
3. **Bring Your Own Vulnerable Driver (BYOVD):** Threat actors drop legitimately signed, vulnerable kernel drivers (e.g., RTCore64.sys, gdrv.sys, mhyprot2.sys). Utilizing arbitrary read/write primitives in these signed drivers, the attacker disables EDR kernel callbacks, zeroes out Defender driver objects, and blinds telemetry without triggering Driver Signature Enforcement (DSE).

| Vector / Mechanism | Live Telemetry (In-Band) | Out-of-Band (OOB) Offline Verification | Tactical Mitigation / Defense |
|---|---|---|---|
| **Kernel Driver / BYOVD** | Get-CimInstance Win32_SystemDriver (Can be spoofed if hooked) | WinPE / Windows Defender Offline pre-boot signature & disk hash verification | HVCI / Core Isolation + Driver Blocklist enforcement |
| **Process Hiding (DKOM)** | Query APIs return empty; thread scheduler still dispatches CPU | Raw volatile memory acquisition (WinPmem / LiME) & pool tag analysis | Hardware-enforced virtualization & Memory Integrity |
| **Userland Preload Hijack** | /etc/ld.so.preload intercepting libc readdir() | Statically linked binaries (busybox) or offline live USB mount | Immutable filesystem attributes (chattr +i), AppArmor |
| **Mobile Memory Implants** | App Store sandbox prevents all process/kernel inspection | MVT forensic backup extraction & ADB USB system state triage | iOS Lockdown Mode + Daily 24h reboot cadence |

# 2. Module 1: Windows Endpoint & Kernel Driver Compromise Assessment

The script windows_compromise_audit.ps1 is engineered for Windows 10/11 and Windows Server 2016-2025. It performs strict, non-destructive heuristic audits of volatile memory artifacts, running drivers, persistence mechanisms, and security log tampering indicators. It generates a full timestamped report directory: .\Windows_Audit_<TIMESTAMP>\.

## Exact Command Execution Syntax:

```
# Open an elevated Administrator PowerShell prompt:
powershell.exe -ExecutionPolicy Bypass -NoProfile -File .\windows_compromise_audit.ps1

# Custom output path execution:
powershell.exe -ExecutionPolicy Bypass -File .\windows_compromise_audit.ps1 -OutputDir C:\IR_Evidence
```

## Heuristic Triage Guide: Interpreting Audit Signals:

| Module Check | [ALERT] Trigger Condition | [WARN] Trigger Condition | Tactical Operator Action |
|---|---|---|---|
| **1. Network Sockets & Listeners** | Unsigned binary OR process located in AppData, Temp, Public, ProgramData holding external socket | High-numbered ephemeral port listening bound to 0.0.0.0 without remote peer | Terminate process, carve binary for sandbox detonation, block remote IP at perimeter firewall. |
| **2. Unsigned Processes** | Binary is unsigned AND executing from user-writable staging directory | Unsigned process outside System32, or signed process running in user staging path | Verify parent process chain. Check hash on VirusTotal. Dump process memory via ProcDump. |
| **3. WMI Fileless Persistence** | Any active CommandLineEventConsumer or ActiveScriptEventConsumer in root\subscription | Unbound __EventFilter without consumer attachment | Extract CommandLineTemplate / ScriptText. Delete consumer via Remove-CimInstance. |
| **4. Registry Persistence & IFEO** | Run key invoking LOLBin (powershell, cmd, mshta) OR any IFEO subkey possessing a Debugger value | Unrecognized 3rd party entry in HKLM/HKCU Run keys | Immediate Sticky Keys / Accessibility backdoor indicator. Delete registry key; isolate host. |
| **5. Kernel Drivers & Services** | CRITICAL: Unsigned running kernel driver (Win32_SystemDriver) or service invoking LOLBins | Non-standard service binary path outside Windows directories | High confidence rootkit / BYOVD implant. Do not reboot live without memory dump. Execute WDO scan. |
| **6. Scheduled Tasks** | Task invoking LOLBins with flags (-enc, Bypass, DownloadString, IEX) | Non-Microsoft task executing from user-writable directory | Unregister task (Unregister-ScheduledTask); inspect staging directory for dropped payloads. |
| **7. Defender Tampering** | DisableRealtimeMonitoring = True OR path/process exclusions configured | Behavior monitoring or script scanning disabled | Purge exclusions (Remove-MpPreference); re-enable real-time and cloud protections. |
| **8. Critical Event Logs** | Event ID 1102 (Security audit log cleared) detected in past 7 days | Event ID 7045 (New service installed) logged in past 7 days | Correlate timestamp with user identity; check SIEM/syslog backups for purged log history. |

## Triggering Windows Defender Offline (WDO) Pre-Boot Rootkit Scan:

When an [ALERT] triggers on an unsigned running kernel driver or suspicious system service, in-band scanning cannot be relied upon to cleanse the system. Windows Defender Offline reboots the machine into a dedicated Windows Recovery Environment (WinRE) container, mounting the disk offline where kernel hooks and active memory filters are inert:

```
# Save all operator work, then execute pre-boot offline scan:
Start-MpWDOScan
# The system will immediately initiate a reboot into the offline WDO environment.
```

## Attack Surface Reduction (ASR) Hardening Rules:

ASR rules intercept malicious behaviors before code execution occurs. Operators must enforce these rules in **Block Mode** (Action = 1) across all managed endpoints:

| ASR Rule Name | Rule GUID | Threat Vector Neutralized |
|---|---|---|
| **Block credential stealing from Windows LSASS** | `9e6c4e1f-7d60-472f-ba1a-a39ef669e4b2` | Mimikatz, procdump, memory scraping of plaintext credentials & NTLM hashes |

| | | |
|---|---|---|
| **Block process creations from PSExec and WMI** | `d1e49aac-8f56-4280-b9ba-993a6d77406c` | Lateral movement via remote WMI process spawning and administrative tools |
| **Block executable content from email & webmail** | `be9ba2d9-53ea-44a7-ac61-759b4f84b0bc` | Phishing attachments, ISO containers, and downloaded script droppers |
| **Block Office apps from creating child processes** | `d4f940ab-401b-4efc-aadc-ad5f3c50688a` | Malicious Word/Excel macros spawning PowerShell, CMD, or WScript |
| **Block untrusted & unsigned processes from USB** | `b2b3f03d-6a65-4f7b-a9c7-1c7ef74a9ba4` | Physical USB drop attacks, autorun worms, untrusted technician binaries |
| **Block persistence through WMI event subscription** | `e6db77e5-3df2-4cf1-b95a-636979351e5b` | Fileless WMI persistence backdoors and script consumers |

```
# PowerShell Script to Enforce Critical ASR Rules in Block Mode:
$asrRules = @(
    '9e6c4e1f-7d60-472f-ba1a-a39ef669e4b2', # LSASS Protection
    'd1e49aac-8f56-4280-b9ba-993a6d77406c', # PSExec/WMI Child Process
    'be9ba2d9-53ea-44a7-ac61-759b4f84b0bc', # Email Executable Content
    'd4f940ab-401b-4efc-aadc-ad5f3c50688a', # Office Child Process Spawning
    'b2b3f03d-6a65-4f7b-a9c7-1c7ef74a9ba4', # USB Unsigned Executables
    'e6db77e5-3df2-4cf1-b95a-636979351e5b'  # WMI Persistence
)
foreach ($guid in $asrRules) {
    Add-MpPreference -AttackSurfaceReductionRules_Ids $guid -AttackSurfaceReductionRules_Actions Enabled
}
```

# 3. Module 2: Rented Web Server (PHP/MySQL VPS/cPanel) Audit & Hardening

Web application hosting environments (cPanel, standard Linux VPS, cloud compute instances) running PHP/MySQL represent the primary attack surface for initial access, credential exfiltration, and SEO spam redirection. The script server_webshell_rootkit_audit.sh executes 8 deep checks across the webroot and host environment.

## Exact Deployment & Execution Instructions:

```
# Step 1: Securely transfer the auditor to the target VPS via SCP:
scp server_webshell_rootkit_audit.sh operator@target-vps.com:/tmp/

# Step 2: SSH into target and execute against target webroot (e.g. /var/www/html or $HOME/public_html):
ssh operator@target-vps.com
chmod +x /tmp/server_webshell_rootkit_audit.sh
/tmp/server_webshell_rootkit_audit.sh /var/www/html

# Step 3: Retrieve generated audit log back to your analysis station:
scp operator@target-vps.com:/tmp/server_security_audit_*.log ./evidence/
```

## Web Application Threat Vectors & Triage Protocol:

- **Obfuscated PHP Primitives:** Attackers evade static regex filters by chaining decompression, character arithmetic, and string transformations (e.g. eval(gzinflate(base64_decode(...))), str_rot13, or variable functions $f = 'sys'.'tem'; $f($_POST['cmd']);). Any match on unconstrained system execution primitives constitutes an immediate [ALERT].
- **Rogue Scripts in Static/Upload Folders:** Legitimate web applications store user media (avatars, PDFs, product images) in dedicated static folders (/uploads/, /media/, /cache/). Threat actors exploit unvalidated file uploads to drop webshells (.phtml, .php5, .phar) into these directories. Any script in an upload folder is high-confidence malicious.
- **.htaccess Trapping & SEO Poisoning:** Apache web servers process directory-level .htaccess directives. Attackers inject php_value auto_prepend_file /tmp/.backdoor.php to automatically execute malicious code before every page load, or inject RewriteCond %{HTTP_REFERER} (google|bing) to redirect search engine bots to scam sites while presenting clean pages to administrators.
- **Userland Rootkits (LD_PRELOAD):** The Linux dynamic linker (ld-linux.so) consults /etc/ld.so.preload before loading any shared object. Rootkits (such as Azazel, Jynx, Reptile userland) hook readdir(), open(), and stat() to hide webshell files from ls and hidden processes from ps. Non-empty /etc/ld.so.preload is a catastrophic compromise.

## Production Hardening Configurations (Copy-Paste Ready):

### 1. Production php.ini Hardening Directives:

```
# Locate active php.ini (e.g. /etc/php/8.2/fpm/php.ini or /etc/php.ini) and configure:
disable_functions = exec,passthru,shell_exec,system,proc_open,popen,curl_exec,curl_multi_exec,parse_ini_file,show_source
session.upload_progress.enabled = 0    # CRITICAL: Neutralizes PHP2Shell LFI-to-RCE race conditions
allow_url_include = Off               # CRITICAL: Prevents Remote File Inclusion (RFI)
allow_url_fopen = Off                 # Restricts remote file streaming unless explicitly required
expose_php = Off                      # Suppresses X-Powered-By HTTP response headers
display_errors = Off                  # Prevents internal path and database credential leaks
open_basedir = /var/www/html/:/tmp/   # Confines PHP file operations strictly to webroot and temp
```

### 2. Apache / cPanel .htaccess Execution Lock for Upload Directories:

Place this .htaccess file inside every public upload folder (e.g. wp-content/uploads/.htaccess). It completely forbids the web server from executing any script handler, rendering uploaded webshells inert:

```
# Prevent direct script execution in static upload folders:

    Require all denied

Options -ExecCGI -Indexes
```

### 3. Linux Host Permission Hardening:

```
# Enforce secure permission baseline across webroot:
find /var/www/html -type d -exec chmod 755 {} +
find /var/www/html -type f -exec chmod 644 {} +
# Lock sensitive database & environment configuration files:
chmod 600 /var/www/html/.env /var/www/html/wp-config.php /var/www/html/configuration.php
```

# 4. Module 3: iPhone (iOS) & Android Mobile Forensic Audit & Hardening

Smartphones represent high-value espionage targets. Mobile spyware cartels deploy zero-click exploits (e.g. BlastPass, ForcedEntry, Subzero) targeting image parsers, WebKit rendering engines, and cellular baseband stacks.

## The Mobile Sandboxing Paradox:

> **[ALERT] THE CONSUMER MOBILE "ANTIVIRUS" FALLACY**
>
> Commercial "Antivirus" or "Security" apps downloaded from the Apple App Store or Google Play Store CANNOT detect kernel-level or state-grade spyware. By design, modern mobile operating systems enforce strict application sandboxing. A userland app has zero visibility into the process table, cannot inspect memory of other applications, cannot read system logs (syslog/logcat), and cannot inspect the kernel. Only external, out-of-band forensic toolkits operating via USB (ADB on Android, unencrypted backups on iOS) can extract and analyze threat indicators.

## Forensic Orchestration via mobile_forensics_audit.sh:

The script mobile_forensics_audit.sh automates threat hunting using Amnesty International's Mobile Verification Toolkit (MVT) combined with deep heuristic device parameter auditing.

```
# Running Android Forensic Audit over USB:
./mobile_forensics_audit.sh android

# Running Apple iOS Forensic Audit over USB (creates full forensic backup):
./mobile_forensics_audit.sh ios

# Auditing an existing pre-acquired iTunes/Finder unencrypted backup:
./mobile_forensics_audit.sh ios /path/to/extracted/ios_backup
```

| Mobile Vector | Diagnostic Inspection Mechanism | Spyware / Threat Indicator | Hardening & Containment Step |
|---|---|---|---|
| **Verified Boot (AVB)** | `adb shell getprop ro.boot.verifiedbootstate` | Status orange, yellow, or red indicates unlocked/tampered bootloader | OEM re-lock or re-flash stock signed firmware image via fastboot |
| **SELinux State** | `adb shell getenforce` | Status Permissive or Disabled (Exploit condition) | Re-flash stock partition; device has been permanently rooted/compromised |
| **Accessibility Services** | `settings get secure enabled_accessibility_services` | Keylogger / Stalkerware hook intercepting screen and keystrokes | Revoke permission immediately; uninstall offending package |
| **Device Administrators** | `dumpsys device_policy` | Anti-uninstall persistence preventing user deletion of spyware | Deactivate device administrator in settings before uninstallation |
| **User Root CAs** | `ls -la /data/misc/user/0/cacerts-added/` | Rogue CA certificates enabling silent HTTPS/TLS proxy MITM | Delete all user-installed certificates in Encryption & Credentials settings |
| **STIX2 IOC Scan** | `mvt-ios check-backup / mvt-android check-adb` | Matches on known NSO Pegasus, Predator, Cytrox C2 domains/hashes | Full hardware retirement / forensic image preservation; rotate all identities |

## Tactical Mobile Hardening Checklist:

1. **Enable Apple iOS Lockdown Mode:** (Settings -> Privacy & Security -> Lockdown Mode). Strictly disables Just-In-Time (JIT) JavaScript compilation, blocks 2G cellular connections, strips complex WebKit font rendering, blocks incoming FaceTime and message attachments from unknown senders, and disables wired USB connections when locked.
2. **The 24-Hour Reboot Cadence:** Most zero-click mobile exploits (e.g. Pegasus) operate as *non-persistent memory-only implants* to evade hardware write protections and avoid bricking devices. Rebooting the device once every 24 hours forces the malware out of memory, forcing the threat actor to re-infect and burn valuable zero-day exploit bandwidth.
3. **Audit User Root Certificates:** Regularly inspect Settings -> Security -> Encryption & Credentials -> User Credentials. Any certificate not explicitly installed by enterprise MDM must be purged immediately.
4. **Disable Sideloading & USB Debugging:** Turn off USB debugging on Android when not actively auditing. Ensure 'Install Unknown Apps' is globally disabled.

# 5. Enterprise Incident Response & Containment Protocol

When an audit script triggers an [ALERT] (such as an unsigned running kernel driver, an active webshell, or an MVT STIX2 IOC match), operators must immediately shift from threat hunting to tactical incident containment. Follow this standardized 6-phase containment protocol:

## Phase 1: Rapid Threat Triage & Verification

Confirm the alert is not a known false positive. Verify file hashes against internal software baselines and VirusTotal. Establish an isolated out-of-band communications channel (Signal / separate hardware) for incident response personnel.

## Phase 2: Immediate Network Isolation (Containment)

Sever all network connectivity to halt lateral movement and data exfiltration. **DO NOT POWER OFF THE HOST IMMEDIATELY**; powering off destroys critical volatile evidence stored in RAM.

```
# Windows Network Isolation (Immediate Host Firewall Lockdown):
netsh advfirewall set allprofiles state on
netsh advfirewall firewall add rule name="IR_BLOCK_ALL_OUT" dir=out action=block
netsh advfirewall firewall add rule name="IR_BLOCK_ALL_IN" dir=in action=block

# Linux VPS Network Isolation (Drop all traffic except forensic SSH if console unavailable):
iptables -P INPUT DROP && iptables -P FORWARD DROP && iptables -P OUTPUT DROP
```

## Phase 3: Volatile Memory & Forensic Evidence Preservation

Extract raw volatile system RAM prior to taking down the system. Memory contains decrypted malware payloads, network sockets, injected DLLs, and plaintext credentials:

```
# Windows Memory Acquisition via WinPmem (executed from external USB drive):
winpmem.exe -o D:\evidence\host_ram_dump.raw

# Linux Memory Acquisition via LiME (Linux Memory Extractor):
insmod lime-$(uname -r).ko "path=/tmp/evidence/linux_ram.lime format=raw"
```

## Phase 4: Scope Assessment & Lateral Movement Investigation

Query domain controllers, perimeter firewalls, and proxy logs for connections to the external IP addresses identified in Module 1. Audit Kerberos ticket granting requests (TGT/TGS) and check for pass-the-hash or token manipulation.

## Phase 5: Enterprise Credential & Identity Revocation

Assume all credentials stored or processed on the compromised host are compromised. Execute a global password and token rotation across Active Directory, cloud providers (AWS/Azure/GCP IAM keys), database connection strings, and SSH authorized keys.

## Phase 6: Golden-Image Eradication & Recovery Workflow

**Never attempt to 'clean' a kernel-compromised or rootkitted operating system while leaving the OS intact.** Eradication must follow the Golden-Image recovery standard:

1. **Storage Zeroization:** Execute complete disk wipe (secure erase or ATA sanitize) to destroy persistence in boot sectors.
2. **Re-Deploy from Known-Clean Baseline:** Reinstall operating system from a cryptographically signed golden image.
3. **Data Restoration:** Restore application and user data exclusively from pre-incident offline cold backups. Inspect all restored files with static scanners before placing in production.
4. **Post-Recovery Surveillance:** Place the restored host in heightened monitoring status for 30 days to detect persistence re-infection attempts.

> **[SUCCESS] POST-INCIDENT POSTURE REINFORCEMENT**
>
> Following incident closure, conduct an executive blameless post-mortem. Update firewall perimeter rules, expand Attack Surface Reduction (ASR) coverage, harden PHP and web server runtime configurations, and integrate continuous automated threat hunting using the toolkit scripts provided in this repository.

# 6. Cyber Defense Remediation Portal & Automated Telemetry Loop

The toolkit includes a local, zero-dependency incident response and remediation portal (portal_server.py). Hosted locally on **http://127.0.0.1:8888**, the portal bridges raw forensic script telemetry with actionable, step-by-step eradication procedures.

## The Automated Telemetry & Auto-Spawn Loop:

1. **Autonomous Portal Startup:** When any audit script (windows_compromise_audit.ps1, server_webshell_rootkit_audit.sh, or mobile_forensics_audit.sh) is executed, it first queries http://127.0.0.1:8888/api/health. If offline, the script automatically spawns portal_server.py in the background without operator intervention.
2. **Triple-Artifact Telemetry Export:** Scans simultaneously generate (1) human-readable reports (Compromise_Assessment_Report.txt), (2) verbatim console execution logs (console_execution.log), and (3) machine-readable structured JSON reports (Compromise_Report.json).
3. **Instant REST Ingestion:** The audit script transmits the report payload via POST /api/upload_report to the local portal, storing findings and console traces in ./portal_reports/.
4. **Browser Auto-Launch:** The script launches the operator's default browser directly into the interactive dashboard.

| Endpoint Route | HTTP Method | Payload / Parameters | Operational Function |
|---|---|---|---|
| `/api/health` | **GET** | None | Health probe used by audit scripts to check if portal is listening |
| `/api/reports` | **GET** | None | Lists all available scans with alert/warning counts and host types |
| `/api/report?id=<id>` | **GET** | id string | Retrieves complete scan details and augmented remediation guide |
| `/api/upload_report` | **POST** | JSON payload (structured or raw_log) | Ingests report, enhances findings with remediation KB, and saves to disk |
| `/` | **GET** | None | Serves the single-page Cyber Defense Incident Response Dashboard |

## Automated Remediation Engine & Risk Meter:

Every ingested finding is matched against the built-in threat remediation knowledge base, providing the operator with:

- **Executive 'What Went Well vs. What Went Wrong' Posture Review:** Categorizes verified defensive baselines (clean drivers, active real-time protection, intact event logs) vs. active threats, blindspots, and script execution warnings in comparative cards.
- **Composite Host Risk Meter (0–100%):** Dynamically weighted based on critical alerts (25 pts each) and warnings (8 pts each). As the operator resolves items, the risk score drops in real time.
- **4-Phase Step-by-Step Fix Guide:** For every anomaly, generates exact commands for Phase 1 (Containment), Phase 2 (Forensic Staging), Phase 3 (Eradication), and Phase 4 (Post-fix Verification).
- **Log & Console Upload with Raw Stream Viewer:** For remote or offline hosts, operators can drag & drop console_execution.log, .txt reports, or .json scans directly into the web UI, toggling the embedded terminal stream at will.

```
# Manual Portal Launch Command:
python portal_server.py
# Portal listens on http://127.0.0.1:8888 with zero external dependencies.
```